

**Moscow Aviation Institute
(National Research University)**

Faculty of Information Technologies and Applied Mathematics

Department of Computational Mathematics and Programming

Laboratory work No. 5, course “Discrete Analysis”

Student: M. A. Khorokhorin
Teacher: A. A. Kukhtichev
Group: M8O-206BV-24
Date: 11.05.2026
Grade:
Signature:

Moscow, 2026

Laboratory work No. 5

Task: Find the longest common substring of two strings using a suffix tree.

Task variant: F. 5, search for the longest common substring.

Time limit	15 seconds
Memory limit	1 GB
Input	standard input or <code>input.txt</code>
Output	standard output or <code>output.txt</code>

Input format. Two strings, one per line.

Output format. The length of the longest common substring on the first line, then all possible common substrings of that length in lexicographic order without duplicates.

Example.

Input:

```
xabay
xabcbay
```

Output:

```
3
bay
xab
```

1 Description

The task is to find all longest common substrings of two strings. To do this, a generalized suffix tree is built for the string $S_1\#S_2\$$, where instead of ordinary separator characters the implementation uses the special codes 256 and 257 (`FIRST_SEPARATOR` and `SECOND_SEPARATOR`). Therefore the service separators cannot coincide with any character of the input strings.

The suffix tree is built by Ukkonen's online algorithm. The transitions of a node are stored in `std::map`, so every step costs $O(\log \sigma)$ and the whole construction takes $O(n \log \sigma)$, where $n = |S_1| + |S_2| + 2$ and $\sigma \leq 258$ is the size of the alphabet. After the construction the tree is traversed. For every node it is determined whether its subtree contains suffixes of the first string and suffixes of the second string. If both kinds of suffixes are present, the string spelled by the path from the root to this node is a common substring. The deepest such node gives the length of the longest common substring.

2 Source code

1 Tree structure

The class `SuffixTree` stores its nodes in a vector. Every node (the private structure `Node`) stores the transitions by symbols (`children`), the suffix link (`suffixLink`) and the bounds of the incoming edge in the combined string (`start`, `end`). For leaves the right end of the edge equals the special value `LEAF_END`; the actual end of a leaf edge is given by the current member `leafEnd_`. New nodes are created by `newNode`, the length of an edge is computed by `edgeLength`, and `child` returns the transition by a symbol or `NONE`.

The alphabet of the tree consists of integers. Ordinary bytes of the input strings fall into the range `[0, 255]`, and the separators have the values 256 and 257. The combined text is built by the function `buildCombinedText`.

2 Ukkonen's algorithm

When the next symbol is added by `extend`, the algorithm maintains the active node `activeNode_`, the active edge `activeEdge_`, the active length `activeLength_` and the number of remaining suffixes `remaining_`. If the required transition is missing, a leaf is created. If the transition exists but the next symbol on the edge differs, the edge is split and an internal node is created. Suffix links make it possible not to start processing every suffix from scratch.

3 Finding the answer

The method `findLcs(separator)` returns the structure `LcsResult` with the length of the answer (`length`) and the sorted list of substrings (`substrings`). Instead of a recursive depth-first search, which overflowed the call stack on inputs such as `aaaa...`, the traversal is iterative and uses an explicit stack. First, the nodes are visited in preorder; for every node its parent and its string depth are recorded. Then the nodes are processed in reverse order, so every node is handled after all its descendants.

For every node a coverage mask is aggregated: whether its subtree contains suffixes of the first string, of the second string, or of both strings. To restore the substring itself, the start position of one suffix of the first string (`firstStart`) is propagated to the parent as well. If an internal node is covered by both strings, its depth is compared with the current maximum, and the corresponding string, built by `makeString`, is added to a `std::set`. The set removes duplicates automatically and sorts the answers lexicographically.

The function `longestCommonSubstrings` builds the tree for two non-empty strings and calls `findLcs`; if one of the strings is empty, the answer has length 0. The function `runLcs`

reads two lines (a missing second line is treated as an empty string, so the program prints 0) and writes the answer. The `main` function only calls `runLcs`.

Listing 1: `include/suffix_tree.hpp`

```
1  #pragma once
2
3  #include <cstdint>
4  #include <map>
5  #include <string>
6  #include <vector>
7
8  inline constexpr int FIRST_SEPARATOR = 256;
9  inline constexpr int SECOND_SEPARATOR = 257;
10
11 struct LcsResult {
12     std::size_t length = 0;
13     std::vector<std::string> substrings;
14
15     bool operator==(const LcsResult& other) const = default;
16 };
17
18 std::vector<int> buildCombinedText(const std::string& first, const std::string& second);
19
20 class SuffixTree {
21 public:
22     explicit SuffixTree(std::vector<int> text);
23
24     [[nodiscard]] LcsResult findLcs(std::size_t separator) const;
25     [[nodiscard]] std::size_t nodeCount() const { return nodes_.size(); }
26
27 private:
28     struct Node {
29         std::map<int, int> children;
30         int suffixLink = 0;
31         int start = 0;
32         int end = 0;
33     };
34
35     int newNode(int start, int end);
36     [[nodiscard]] int edgeLength(int node) const;
37     [[nodiscard]] int child(int node, int symbol) const;
38     void extend(int position);
39     [[nodiscard]] std::string makeString(int start, int length) const;
40
41     std::vector<int> text_;
42     std::vector<Node> nodes_;
43     int leafEnd_ = -1;
44     int activeNode_ = 0;
45     int activeEdge_ = 0;
46     int activeLength_ = 0;
47     int remaining_ = 0;
48 };
```

Listing 2: `src/suffix_tree.cpp`

```
1  #include "suffix_tree.hpp"
2
3  #include <cstdint>
```

```

4  #include <ranges>
5  #include <set>
6  #include <string>
7  #include <utility>
8  #include <vector>
9
10 namespace {
11
12 constexpr int ROOT = 0;
13 constexpr int NONE = -1;
14 constexpr int LEAF_END = -1;
15
16 constexpr unsigned COVERS_FIRST = 1U;
17 constexpr unsigned COVERS_SECOND = 2U;
18 constexpr unsigned COVERS_BOTH = COVERS_FIRST | COVERS_SECOND;
19
20 }
21
22 std::vector<int> buildCombinedText(const std::string& first, const std::string& second) {
23     std::vector<int> result;
24     result.reserve(first.size() + second.size() + 2);
25     for (const unsigned char symbol : first) {
26         result.push_back(symbol);
27     }
28     result.push_back(FIRST_SEPARATOR);
29     for (const unsigned char symbol : second) {
30         result.push_back(symbol);
31     }
32     result.push_back(SECOND_SEPARATOR);
33     return result;
34 }
35
36 SuffixTree::SuffixTree(std::vector<int> text) : text_(std::move(text)) {
37     nodes_.reserve(text_.size() * 2 + 1);
38     newNode(NONE, NONE);
39     for (int position = 0; position < static_cast<int>(text_.size()); ++position) {
40         extend(position);
41     }
42 }
43
44 int SuffixTree::newNode(int start, int end) {
45     nodes_.push_back(Node{.children = {}, .suffixLink = 0, .start = start, .end = end});
46     return static_cast<int>(nodes_.size()) - 1;
47 }
48
49 int SuffixTree::edgeLength(int node) const {
50     const int edgeEnd = nodes_[node].end == LEAF_END ? leafEnd_ : nodes_[node].end;
51     return edgeEnd - nodes_[node].start + 1;
52 }
53
54 int SuffixTree::child(int node, int symbol) const {
55     const auto iterator = nodes_[node].children.find(symbol);
56     return iterator == nodes_[node].children.end() ? NONE : iterator->second;
57 }
58
59 void SuffixTree::extend(int position) {
60     leafEnd_ = position;
61     ++remaining_;
62     int lastInternal = NONE;
63
64     while (remaining_ > 0) {
65         if (activeLength_ == 0) {

```

```

66     activeEdge_ = position;
67 }
68
69     const int edgeSymbol = text_[activeEdge_];
70     const int next = child(activeNode_, edgeSymbol);
71
72     if (next == NONE) {
73         nodes_[activeNode_].children[edgeSymbol] = newNode(position, LEAF_END);
74         if (lastInternal != NONE) {
75             nodes_[lastInternal].suffixLink = activeNode_;
76             lastInternal = NONE;
77         }
78     } else {
79         const int nextLength = edgeLength(next);
80         if (activeLength_ >= nextLength) {
81             activeEdge_ += nextLength;
82             activeLength_ -= nextLength;
83             activeNode_ = next;
84             continue;
85         }
86
87         const int splitSymbol = text_[nodes_[next].start + activeLength_];
88         const int currentSymbol = text_[position];
89         if (splitSymbol == currentSymbol) {
90             ++activeLength_;
91             if (lastInternal != NONE) {
92                 nodes_[lastInternal].suffixLink = activeNode_;
93             }
94             break;
95         }
96
97         const int split = newNode(nodes_[next].start, nodes_[next].start + activeLength_ - 1);
98         nodes_[activeNode_].children[edgeSymbol] = split;
99         const int leaf = newNode(position, LEAF_END);
100         nodes_[split].children[currentSymbol] = leaf;
101         nodes_[next].start += activeLength_;
102         nodes_[split].children[splitSymbol] = next;
103
104         if (lastInternal != NONE) {
105             nodes_[lastInternal].suffixLink = split;
106         }
107         lastInternal = split;
108     }
109
110     --remaining_;
111
112     if (activeNode_ == ROOT && activeLength_ > 0) {
113         --activeLength_;
114         activeEdge_ = position - remaining_ + 1;
115     } else if (activeNode_ != ROOT) {
116         activeNode_ = nodes_[activeNode_].suffixLink;
117     }
118 }
119 }
120
121 std::string SuffixTree::makeString(int start, int length) const {
122     std::string result;
123     result.reserve(static_cast<std::size_t>(length));
124     for (int i = 0; i < length; ++i) {
125         result.push_back(static_cast<char>(text_[start + i]));
126     }
127     return result;

```

```

128 }
129
130 LcsResult SuffixTree::findLcs(std::size_t separator) const {
131     const int separatorIndex = static_cast<int>(separator);
132     const int lastIndex = static_cast<int>(text_.size()) - 1;
133     const std::size_t count = nodes_.size();
134
135     std::vector<int> order;
136     order.reserve(count);
137     std::vector<int> parent(count, NONE);
138     std::vector<int> depth(count, 0);
139     std::vector<int> stack = {ROOT};
140     while (!stack.empty()) {
141         const int node = stack.back();
142         stack.pop_back();
143         order.push_back(node);
144         for (const auto& [symbol, next] : nodes_[node].children) {
145             parent[next] = node;
146             depth[next] = depth[node] + edgeLength(next);
147             stack.push_back(next);
148         }
149     }
150
151     std::vector<unsigned> coverage(count, 0U);
152     std::vector<int> firstStart(count, NONE);
153     int bestLength = 0;
154     std::set<std::string> found;
155
156     for (const int node : std::views::reverse(order)) {
157         if (nodes_[node].children.empty()) {
158             const int suffixStart = leafEnd_ + 1 - depth[node];
159             if (suffixStart < separatorIndex) {
160                 coverage[node] = COVERS_FIRST;
161                 firstStart[node] = suffixStart;
162             } else if (suffixStart > separatorIndex && suffixStart < lastIndex) {
163                 coverage[node] = COVERS_SECOND;
164             }
165         } else if (node != ROOT && coverage[node] == COVERS_BOTH && firstStart[node] != NONE &&
166             firstStart[node] + depth[node] <= separatorIndex && depth[node] >= bestLength) {
167             if (depth[node] > bestLength) {
168                 bestLength = depth[node];
169                 found.clear();
170             }
171             found.insert(makeString(firstStart[node], depth[node]));
172         }
173
174         const int up = parent[node];
175         if (up != NONE) {
176             coverage[up] |= coverage[node];
177             if (firstStart[up] == NONE) {
178                 firstStart[up] = firstStart[node];
179             }
180         }
181     }
182
183     return LcsResult{.length = static_cast<std::size_t>(bestLength),
184         .substrings = std::vector<std::string>(found.begin(), found.end())};
185 }

```

Listing 3: include/lcs.hpp

```
1 | #pragma once
2 |
3 | #include <iosfwd>
4 | #include <string>
5 |
6 | #include "suffix_tree.hpp"
7 |
8 | LcsResult longestCommonSubstrings(const std::string& first, const std::string& second);
9 | void runLcs(std::istream& input, std::ostream& output);
```

Listing 4: src/lcs.cpp

```
1 | #include "lcs.hpp"
2 |
3 | #include <istream>
4 | #include <ostream>
5 | #include <string>
6 |
7 | #include "suffix_tree.hpp"
8 |
9 | namespace {
10 |
11 | std::string readLine(std::istream& input) {
12 |     std::string line;
13 |     if (!std::getline(input, line)) {
14 |         return "";
15 |     }
16 |     if (!line.empty() && line.back() == '\r') {
17 |         line.pop_back();
18 |     }
19 |     return line;
20 | }
21 |
22 | }
23 |
24 | LcsResult longestCommonSubstrings(const std::string& first, const std::string& second) {
25 |     if (first.empty() || second.empty()) {
26 |         return LcsResult{};
27 |     }
28 |     const SuffixTree tree(buildCombinedText(first, second));
29 |     return tree.findLcs(first.size());
30 | }
31 |
32 | void runLcs(std::istream& input, std::ostream& output) {
33 |     const std::string first = readLine(input);
34 |     const std::string second = readLine(input);
35 |     const LcsResult result = longestCommonSubstrings(first, second);
36 |     output << result.length << '\n';
37 |     for (const std::string& substring : result.substrings) {
38 |         output << substring << '\n';
39 |     }
40 | }
```

Listing 5: app/main.cpp

```
1 | #include <iostream>
2 |
3 | #include "lcs.hpp"
4 |
5 | int main() {
6 |     std::ios::sync_with_stdio(false);
7 |     std::cin.tie(nullptr);
8 |
9 |     runLcs(std::cin, std::cout);
10 |    return 0;
11 | }
```

3 Console

```
# make build PRESET=release
# build/release/lab5/lab5_main
xabay
xabcbay
3
bay
xab
```

4 Performance test

1 Method

The test compares the construction of the generalized suffix tree together with the search for the answer against the classical dynamic programming solution of the longest common substring problem. The DP keeps only two rolling rows of length $|S_2| + 1$ and runs in $O(|S_1| \cdot |S_2|)$ time and $O(|S_2|)$ memory. The suffix tree is built in $O((|S_1| + |S_2|) \log \sigma)$, where $\sigma \leq 258$ is the size of the alphabet, after which a single iterative traversal is performed.

On small sizes the benchmark compares the full answer sets of the two algorithms. On large sizes the DP is skipped (when $(|S_1| + 1)(|S_2| + 1)$ exceeds $25 \cdot 10^6$ cells), since its quadratic running time becomes too large.

2 Results

$ S_1 = S_2 $	Suffix tree, μs	DP, μs	Check
1 000	996	4 847	yes
2 000	1 329	24 298	yes
4 000	3 482	106 665	yes

Table 1: Comparison with dynamic programming, 4-letter alphabet

$ S_1 = S_2 $	Suffix tree, μs	Answer length
10 000	12 783	12
50 000	208 406	15
100 000	339 247	17

Table 2: Large tests, DP skipped

```
# make build PRESET=release
# python3 lab5/benchmark/generator.py 1000 1000 4 42
# build/release/lab5/lab5_benchmark < lab5/benchmark/tests.txt
Suffix tree LCS time: 996us
LCS length: 10
LCS count: 1
DP LCS time: 4847us
DP LCS length: 10
DP LCS count: 1
```

Results equal: yes

```
# python3 lab5/benchmark/generator.py 4000 4000 4 42
# build/release/lab5/lab5_benchmark < lab5/benchmark/tests.txt
Suffix tree LCS time: 3482us
LCS length: 12
LCS count: 1
DP LCS time: 106665us
DP LCS length: 12
DP LCS count: 1
Results equal: yes
```

```
# python3 lab5/benchmark/generator.py 100000 100000 4 42
# build/release/lab5/lab5_benchmark < lab5/benchmark/tests.txt
Suffix tree LCS time: 339247us
LCS length: 17
LCS count: 1
DP LCS skipped: input is too large
```

3 Analysis of the results

Already on small tests the suffix tree is faster than the DP: for strings of length 4000 the speedup is about **30 times**. As the size grows further, the DP becomes impractical because of its quadratic running time, while the suffix tree keeps working within the limits. The benchmark also checks correctness: on small tests the sets of substrings found by the tree and by the DP coincide.

5 Work diary

- 11.05.2026: studied the task of finding the longest common substring of two strings using a suffix tree.
- 11.05.2026: implemented Ukkonen's algorithm for the combined string with two unique separators that do not occur in the input alphabet.
- 11.05.2026: added a tree traversal with masks of suffix membership in the first and second strings; the answers are collected in a `std::set`.
- 11.05.2026: wrote the generator and the benchmark; the result of the suffix tree is checked against dynamic programming on feasible sizes.

6 Conclusions

By completing this laboratory work for the course “Discrete Analysis”, I implemented the search for the longest common substring of two strings using a generalized suffix tree. The main part of the work was the implementation of Ukkonen’s online algorithm: the active point, the counter of unprocessed suffixes, edge splitting and suffix links.

To find the answer, the tree is traversed iteratively with an explicit stack, and every node is marked according to the presence of suffixes of the first and second strings in its subtree. The deepest node covered by both strings gives the length of the answer, and `std::set` ensures that the output has no duplicates and is in lexicographic order.

Testing confirmed the advantage of the suffix tree over dynamic programming: on strings of length 4 000 the tree is about **30 times** faster, and on larger sizes the DP is no longer used because of its quadratic running time.

References

- [1] Ukkonen, E. *On-line construction of suffix trees*. Algorithmica, 14(3):249–260, 1995.
- [2] Gusfield, D. *Algorithms on Strings, Trees, and Sequences*. Cambridge University Press, 1997.
- [3] Cormen T. H., Leiserson C. E., Rivest R. L., Stein C. *Introduction to Algorithms, 2nd edition*. — MIT Press, 2001.